System Design: Tokenization & Inverted Index

At the core of every modern search engine (like Google, Elasticsearch, or Apache Lucene) lies a data structure called the **Inverted Index**. To build this index, raw text must first go through a text-processing pipeline called **Tokenization**. This cheat sheet is optimized for high-yield system design interview prep, minimizing wasted space.

1. The Core Concepts (In Simple Terms)

CONCEPT 1

What is Tokenization?

Imagine reading a sentence and chopping it up into individual, meaningful words. Tokenization is the process of breaking down a stream of raw text into smaller chunks called **tokens** (usually words or terms).

A production-grade pipeline does more than just split by spaces:

1. **Lowercasing:** `"Apple"` → `"apple"`
2. **Stripping:** Removing punctuation `"hello!"` → `"hello"`
3. **Stop-Words:** Dropping common words ("the", "is") to save massive amounts of space.
4. **Stemming:** Reducing words to their root form (e.g., `"running"` → `"run"`) so tense doesn't break search.

CONCEPT 2

What is an Inverted Index?

Think of the index at the very back of a textbook. Instead of reading the whole book to find a topic, you look up the specific word, and it gives you the exact page numbers.

An inverted index flips the traditional database relationship:

- **Forward Index:** Document → Words
- **Inverted Index:** Word → Documents

It maps a specific *token* to its *Postings List* (the array of Document IDs where that token appears).

2. Step-by-Step Data Flow Example

Let's process two incoming documents to see how the data transforms:

- **Doc 1:** `The quick brown foxes jump.`
- **Doc 2:** `A quick brown fox is fast.`

A. Tokenization Pipeline Output

Doc ID	Raw Text Split	After Pipeline (Lower, Stem, Remove Stops)
1	[The, quick, brown, foxes, jump.]	`quick`, `brown`, `fox`, `jump`
2	[A, quick, brown, fox, is, fast.]	`quick`, `brown`, `fox`, `fast`

B. The Resulting Inverted Index

Term (Token)	Postings List (Doc IDs)	System Design Implication
brown	[1, 2]	Allows O(1) term lookup to immediately find all matching documents.
fast	[2]	Postings are kept **sorted** to allow highly efficient list intersections.
fox	[1, 2]	"foxes" became "fox". A search for either word will hit both documents.
jump	[1]	The Term Dictionary is often kept in RAM (using Tries/ FSTs).
quick	[1, 2]	The bulky Postings Lists are kept on disk and loaded dynamically.

3. System Design Interview Focus Points

In an interview, defining the terms isn't enough. You must discuss scale, storage efficiency, and distributed processing. Here are the three most critical concepts you must bring up.

1. Index Compression (Storage Optimization)

Postings lists for common words can contain millions of Doc IDs. Storing them as raw 32-bit or 64-bit integers wastes massive amounts of memory and disk I/O. Interviewers expect you to know how to compress them:

- **Delta Encoding:** Because posting lists are sorted, you don't store absolute IDs (e.g., `[1000, 1005, 1012]`). Instead, you store the differences/deltas: `[1000, 5, 7]`. This keeps the numbers extremely small.
- **Variable Byte Encoding (or Elias-Fano):** Compress those small delta numbers into fewer bytes. Instead of using 4 bytes for the number '5', Variable Length Encoding uses just 1 byte. This can shrink index size by 70%+.

2. Sharding the Index (Scalability)

When your index exceeds a single machine's capacity, how do you distribute it?

Document-Partitioned (Industry Standard)
Docs are hashed to a specific shard. Each shard builds a full index for *only its subset of documents*.
Trade-off: High write throughput. However, reads require a *Scatter-Gather* approach (query is broadcast to all shards, and results are merged at an aggregator). Tools like Elasticsearch use this.

Term-Partitioned (Theoretical)
Sharded based on the word (e.g., words A-M on Node 1, N-Z on Node 2).
Trade-off: Highly efficient reads (a query for "apple" only hits Node 1). But writes are terrible—adding one document requires sending network requests to almost every node in the cluster. Rarely used in modern engines.

3. Handling High Write Throughput (Mutability)

Updating an existing inverted index in-place on disk is horribly inefficient (you'd have to constantly shift massive arrays of integers to insert a new Doc ID in sorted order).

- **Immutable Segments:** Modern engines don't update files. New documents are indexed in RAM. Periodically, RAM is flushed to disk as a brand new, immutable "Segment" (a mini inverted index).
- **Background Merging:** To prevent having thousands of tiny segments to search through, a background process continually merges smaller segments into larger ones (similar to how Log-Structured Merge / LSM trees operate).